endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 7

AGENDA CURS 7

1. Definirea de seturi statice de valori cu Enum
2. Primitive ca obiecte: wrapper classes
3. Tipuri generice
4. Collections Framework
5. Stocare valori ordonate cu implementari de List
6. Stocare valori unice cu implementari de Set
7. Stocare perechi cheie-valoare cu implementari de Map

ENUMS

- Un **enum** este un tip special de clasa care reprezinta un set fix de valori constante
- Este util cand ai un numar limitat de optiuni posibile (ex: zilele saptamanii, stari ale unei comenzi, tipuri de utilizatori etc.).
- Un enum poate avea: atribute, constructori, metode.
- Un enum poate implementa interfete, deci poti folosi polimorfism.
- Metode utile:
  - values() – returneaza un array cu toate valorile
  - valueOf(String name) – returneaza enum-ul corespunzator numelui
  - ordinal() – pozitia in care a fost declarat in enum (incepand de la 0)
- Beneficii:
  - Cod mai clar si mai sigur
  - Verificare a tipului de data la compile time
  - Se poate folosi in switch
  - Poate avea logica proprie

```java
public enum OrderStatus {
    NEW("New order"),
    PROCESSING("Order is being processed"),
    SHIPPED("Order has been shipped");

    private String description;

    OrderStatus(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }
}
```

```java
OrderStatus status = OrderStatus.PROCESSING;
System.out.println(status.getDescription());
```


endava

WRAPPER CLASSES

- **Wrapper classes** sunt clase care „împachetează” tipurile de date primitive în obiecte.
- Sunt utile in combinatie cu tipurile generice, deoarece tipurile generice functioneaza doar cu tipuri de Object, nu si cu primitive.
- **Autoboxing**: Conversia automată a unui tip primitiv într-un obiect wrapper.

```
int x = 5;
Integer y = x;  // autoboxing
```

- **Unboxing**: Conversia automată a unui obiect wrapper în tip primitiv.

```
Integer a = 10;
int b = a;  // unboxing
```

Tip primitiv	Wrapper class
`byte`	`Byte`
`short`	`Short`
`int`	`Integer`
`long`	`Long`
`float`	`Float`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`

TIPURI GENERICE

- **Tipurile generice** permit scrierea de cod care functioneaza cu mai multe tipuri de date, intr-un mod sigur si reutilizabil.
- Permite programatorii sa isi exprime intentia de a folosi tipuri specifice, iar compilatorul sa se asigure ca aceste tipuri sunt corecte.
- Beneficii:
  - Prevenire erori la runtime (prin verificare la compilare)
  - Eliminarea cast-urile explicite
  - Cod mai clar si reutilizabil

TIPURI GENERICE - CLASE

- **O clasa generica** este o clasa care foloseste cel putin un tip generic.
- Un **type parameter** un tip generic simbolizat de regula de o singura litera mare (ex: <T>, <V>, <E>)
- Un type parameter este un placeholder pentru tipul de data al unui:
  - atribut
  - parametru de metoda
  - tip returnat de metoda
- Cand se creeaza o instanta a unei clase generice, trebuie sa fie specificat tipul concret care va inlocui tipul generic definit de clasa.
- Nu putem folosi un wildcard pentru a define o clasa generica.
- Type erasure: folosirea instantelor de clase generice fara a specifica tipul concret – **nu este recomandat.**

```java
public class Box<T> {
    private T content;

    public void set(T content) {
        this.content = content;
    }

    public T get() {
        return content;
    }
}
```

```java
Box<String> stringBox = new Box<>();
stringBox.set("Hello");
String message = stringBox.get(); // No cast needed
```


endava

TIPURI GENERICE – METODE STATICE

- Pentru metodele statice, tipul generic poate fi folosit pentru a define tipul de data:
  - al unui parametru
  - al tipului returnat
- Tipul generic trebuie declarat ca parte din semnatura metodei, inainte de tipul returnat.
- Cand o metoda generica returneaza un tip generic, ar trebui sa folosim un type parameter in locul unui wildcard.
- Clasa in care se afla metoda statica generica nu trebuie sa fie generica.

```
public class Utils {
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.println(element);
        }
    }
}
```

```
String[] names = { "Alice", "Bob" };
Utils.printArray(names);

Integer[] numbers = { 1, 2, 3 };
Utils.printArray(numbers);
```

TIPURI GENERICE - BOUNDS

- Pentru a restrictiona tipurile concrete care pot fi folosite pentru un tip generic, se pot folosi **bounded type parameters**.
- Upper bounded type parameter
  - extends

```java
public class NumberBox<T extends Number> {
    private T number;

    public void set(T number) {
        this.number = number;
    }

    public double getDoubleValue() {
        return number.doubleValue();
    }
}
```


endava

TIPURI GENERICE - WILDCARDS

- Un **wildcard** reprezinta un tip de data necunoscut.
- Poate fi folosit cand vrem sa ne referim la un tip generic, dar nu avem nevoie sa stim tipul specific.
- Este util pentru metode care lucreaza cu o mare varietate de tipuri de obiecte, dar nu modifica starea obiectelor.
- Upper bounded wildcards
  - extends
- Lower bounded wildcards
  - super

```java
public void printList(List<?> list) {
    for (Object item : list) {
        System.out.println(item);
    }
}
```

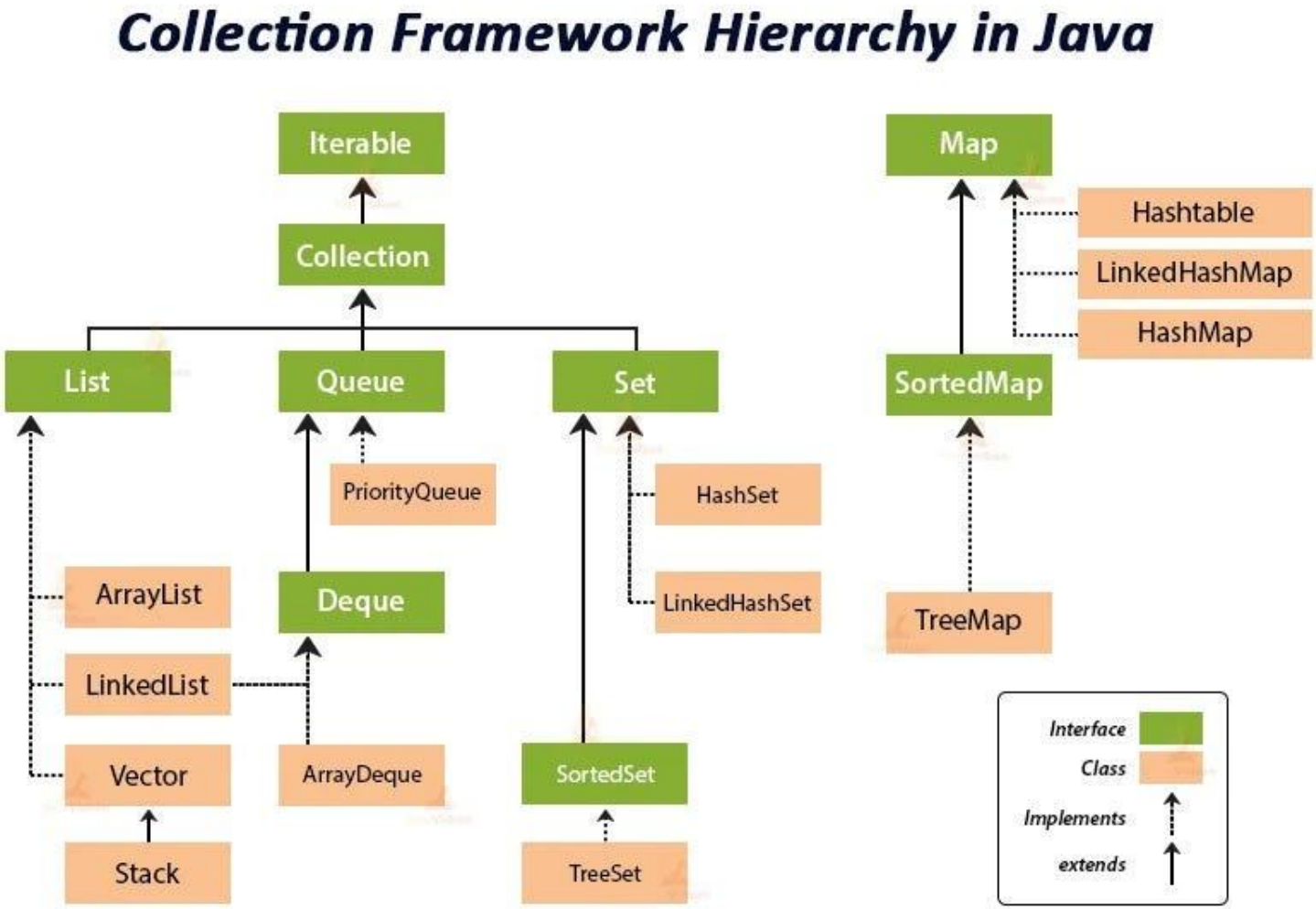
```java
public void printNumbers(List<? extends Number> numbers) {
    for (Number n : numbers) {
        System.out.println(n);
    }
}
```

```java
public void addIntegers(List<? super Integer> list) {
    list.add(42); // OK
}
```

TIPURI GENERICE

Categorie	`<T>` – **Type Parameter**	`<?>` – **Wildcard (tip necunoscut)**
Definitie	`<T>` defineste un **type parameter** specific care poate fi folosit in intreaga clasa sau metoda.	`<?>` este un wildcard care reprezinta un tip necunoscut si poate fi folosit doar unde tipul nu conteaza.
Flexibilitate	`<T>` permite crearea de instante ale tipului generic si este folosit des pentru definirea de tipuri generice noi.	`<?>` este folosit in principal pentru parametri de metoda sau tipuri returnate, cand vrei sa accepti sau sa returnezi orice tip fara sa-l modifici.
Siguranta tipului	Cu `<T>`, ai control total asupra tipului, ceea ce asigura siguranta cand manipulezi obiectele.	Cu `<?>`, poti doar sa citesti din acel tip generic, nu sa-l modifici – ceea ce este sigur, dar limitativ ca functionalitate.

COLLECTIONS FRAMEWORK



COLLECTION<E>

- Interfata **Collection** este radacina majoritatii structurilor de date definite in Collections Framework.
- Defineste metode pentru operatii frecvent folosite: add(), remove(), contains(), size() etc.
- Este o interfata generica: Collection<E>

LIST<E>

- Interfata List este o interfata care mosteneste Collection.
- Reprezinta o colectie ordonata de elemente.
- Caracteristici:
  - Ordinea este mentinuta.
  - Permite elemente duplicate.
  - Poate fi modificata (adaugare, stergere, inlocuire).
  - Acces random prin index (in special in ArrayList).

Metoda	Descriere
`add(E element)`	Adauga un element la final
`add(int index, E element)`	Adauga un element la o anumita pozitie
`get(int index)`	Returneaza elementul de la index
`set(int index, E element)`	Inlocuieste elementul la un index
`remove(int index)`	Sterge elementul de la index
`indexOf(Object o)`	Returneaza primul index al elementului
`lastIndexOf(Object o)`	Returneaza ultimul index al elementului
`subList(int from, int to)`	Returneaza o portiune din lista
`size()`	Numarul de elemente

LIST<E>

- Implementari uzuale:

Clasa	Acces rapid	Inserare usoara	Thread-safe	Use-case-uri recomandate
`ArrayList`	✅ O(1)	❌ la inceput	❌	Liste de date, afisare, acces frecvent
`LinkedList`	❌ O(n)	✅ la capete	❌	Cozi, liste cu modificari frecvente
`Vector`	✅ O(1)	❌	✅	Aplicatii vechi, nevoi simple multi-thread

SET<E>

- Interfata Set este o interfata care mosteneste Collection.
- Reprezinta o colectie de elemente unice.
- Caracteristici:
  - Nu permite duplicate – adaugarea unui element deja existent nu are efect.
  - Ordinea nu este garantata (depinde de implementare)
  - Permite operatii matematice: reuniune, intersectie, diferenta (folosind metode precum retainAll, addAll, removeAll).

Metoda	Descriere
`add(E element)`	Adauga un element daca nu exista deja
`remove(Object o)`	Sterge un element
`contains(Object o)`	Verifica daca elementul exista
`isEmpty()`	Verifica daca setul este gol
`size()`	Numarul de elemente
`iterator()`	Iterare prin set

endava

SET<E>

- Implementari uzuale:

Clasa	Ordine mentinuta	Permite `null`	Thread-safe	Use-case-uri recomandate
`HashSet`	❌ (neordonat)	✅	❌	Eliminare de duplicate, performanta generala
`LinkedHashSet`	✅ (ordine inserare)	✅	❌	Colectii unice unde conteaza ordinea
`TreeSet`	✅ (sortat)	❌*	❌	Liste unice sortate, cautari rapide in interval

> * `TreeSet` arunca `NullPointerException` daca incerci sa adaugi `null` si exista comparator sau sortare implicita.

endava

MAP<K, V>

- Interfata Map este o colectie care **asociaza chei cu valori**. Nu mosteneste interfata Collection.
- Caracteristici:
  - Fiecare cheie este unica – daca adaugi din nou aceeasi cheie, valoarea veche este inlocuita.
  - Valorile pot fi duplicate.
  - Nu garanteaza ordinea (depinde de implementare)

Metoda	Descriere
`put(K key, V value)`	Adauga sau actualizeaza o pereche cheie-valoare
`get(Object key)`	Returneaza valoarea asociata cheii
`remove(Object key)`	Sterge o intrare pe baza cheii
`containsKey(Object key)`	Verifica daca cheia exista
`containsValue(Object v)`	Verifica daca valoarea exista
`keySet()`	Returneaza un `Set` cu toate cheile
`values()`	Returneaza o `Collection` cu toate valorile
`entrySet()`	Returneaza un `Set` cu perechile cheie-valoare

MAP<K, V>

- Implementari uzuale:

Clasa	Ordine mentinuta	Permite `null`	Thread-safe	Use-case-uri recomandate
`HashMap`	❌	✅ (1 cheie, valori nelimitat)	❌	Stocare generala key-value, cautare rapida
`LinkedHashMap`	✅ (ordine inserare)	✅	❌	Cache-uri simple, mentinere ordine
`TreeMap`	✅ (sortat dupa cheie)	❌ chei `null`	❌	Dictionare sortate, intervale de cautare
`Hashtable`	❌	❌	✅	Cod legacy, aplicatii vechi, sincronizare simpla

endava

Q&A

THANK YOU

DANIELA SPILCĂ